

Reflection: Deploying PyTorch ML Workloads with Docker & Kubernetes

Madhava Ketha

August 29, 2026

This project presented a comprehensive journey through the MLOps lifecycle, bridging the gap between theoretical model development and production-grade deployment. The most significant challenges emerged not during the PyTorch model construction, but at the integration points between the host environment, Docker containers, and Kubernetes orchestration.

Initially, configuring the local PyTorch environment for GPU acceleration posed a hurdle. Despite having dedicated hardware, the standard PyTorch installation defaulted to the CPU. Resolving this required forcing a re-installation using the specific CUDA 12.1 wheel index to ensure the host could properly offload tensor computations. This early roadblock highlighted the importance of strict dependency management, which became crucial when designing the multi-stage Dockerfiles. Ensuring the training image was robust enough to compile operations while keeping the serving image lightweight required careful version pinning in the isolated `requirements.txt` files.

Attempting to run these GPU-accelerated PyTorch workloads locally via Kubernetes further underscored the complexities of hardware virtualization. Deploying containerized GPU tasks through Minikube and Docker Desktop on Windows revealed deep abstraction roadblocks, such as the NVML initialization failure (`Failed to initialize NVML: Not Supported`) caused by container runtime restrictions and translation barriers between Windows, WSL2, and Kubernetes nodes. Transitioning through these constraints demonstrated that while local Kubernetes orchestration excels for CPU-based testing, validating hardware-accelerated workloads often requires native Linux-based runtime toolkits or direct container execution paths.

The transition to Kubernetes introduced networking and state management complexities. When deploying the serving layer, I encountered a `RemoteDisconnected` error during local inference testing. Debugging revealed two cascading issues. First, the `kubect1 port-forward` mapping had to precisely match the FastAPI internal port. Second, crafting the multipart/form-data HTTP request from the host required strict boundary handling; relying on native PowerShell cmdlets proved fragile, whereas a dedicated Python `requests` script successfully transmitted the image payloads to the prediction endpoint.

Finally, orchestrating the PersistentVolumeClaims (PVCs) to share the model checkpoint between the ephemeral Training Job and the long-running Serving Deployment required precise YAML configuration. Ensuring the training job successfully wrote the `.pt` checkpoint to the mounted volume before the serving replica attempted to load it reinforced the necessity of Kubernetes readiness and liveness probes.

Ultimately, standardizing the Git branching strategy and CI/CD pipelines early on provided a reliable safety net. By isolating infrastructure changes into feature branches,

it became much easier to isolate and debug container crashes, making the end-to-end deployment of the ResNet-18 model a robust, repeatable process.